Alibaba Travel Ticket Reservation System – Phase 2

Alireza Nobakht **Mohammad Taghi Ghaffari** **Mahan Nojavan**

🔧 System Overview

This project builds upon the core travel reservation system by integrating advanced analytical functionality and performance optimization features. Users interact with the system to reserve, cancel, and report issues with travel tickets across three transportation modes: bus, train, and plane. Administrative users (support staff) can view complaints, track cancellations, and access dashboards. Phase 2 adds functionality for data-driven insights to support business decisions, as well as efficient data retrieval via indexing and stored procedures.

📐 Database Design Summary

Entity-Relationship Design Enhancements

Relationships established between ticket, vehicle, and mode-specific tables like bus, plane, train through polymorphic join logic. Each ticket is connected to a reservation, which is linked to a passenger derived from the general person table. Report table tracks user-submitted issues against tickets, while support staff respond and resolve them.

Normalization to 3NF

All attributes are fully dependent on the primary key. Transitive dependencies removed by segregating passenger, support, and sponsor from the main person table. Referential integrity is ensured with appropriate ON DELETE and ON UPDATE constraints.

🗂 Tables and Indexing Strategy

A detailed indexing strategy was implemented as follows:

Table	Primary Key	Indexed Columns	Purpose
person	person_id	email, phone_number, city	Search by identity & geography

ticket	ticket_id	vehicle_id, reservation_id, created_at	Fast filtering by vehicle/date/reservation
reservation	reservation_id	passenger_id, status	Cancellations & passenger history
vehicle	vehicle_id	vehicle_type	Fast access to transport types
report	report_id	ticket_id, report_subject	Grouping by complaint type
bus, train, plane	vehicle_id (FK)	(inherit from vehicle)	Dynamic join based on mode

Indexing Notes:

- Multi-column indexing used for composite search filters.
- Temporal data such as created_at indexed for reporting.
- ENUM data types used to restrict status and mode types.

⚙️ Analytical & Operational Queries Implemented (22+ Total)

Some advanced queries implemented:

1. **Monthly Aggregated Payments Per User**
2. **Users With Single Ticket Per City (via GROUP BY and HAVING COUNT=1)**
3. **Latest Ticket Buyer (ORDER BY created_at DESC LIMIT 1)**
4. **Users With Total Payments Above Average (Using Subquery AVG)**
5. **Top 3 Ticket Buyers in Last 7 Days (Temporal Filtering)**
6. **Number of Tickets by Transportation Type**
7. **Tickets Sold by City in Tehran Province**
8. **2nd Best-Selling Ticket in Ticket Bin (Using DENSE_RANK)**
9. **Support Staff With Highest Cancellations (Including % Computation)**

🧠 Stored Procedures and Functions

Procedure	Input	Output	Purpose
sp_user_tickets_ordered	email/phone	ticket list	Ticket history ordered by time
sp_support_cancelled_users	email/phone	user list	Users whose reservations were cancelled
sp_city_tickets	city name	ticket list	Tickets purchased in a city
sp_phrase_search	text	matched tickets	Searches name/route/class
sp_city_users	user contact	users in same city	Localized user discovery
sp_top_n_buyers	date + n	top buyers	Frequent buyers since date
sp_cancelled_by_mode	mode ENUM	canceled ticket list	Filtered by transportation type
sp_report_topic_users	topic	user list	Top reporters by subject

Each procedure includes:

- Parameter sanitization
- Performance-aware joins
- Use of indexing-aware WHERE conditions
- ENUM enforcement (e.g. transport type)

🔍 Query Optimization Techniques Applied

Technique	Description
Indexing	On search-heavy fields like dates, city, transport mode
ENUM Columns	For status, transport_type to avoid string matching
Subqueries with HAVING	Used in aggregates like one-ticket-per-city users
Joins Optimized via WHERE	Avoid Cartesian joins; explicitly restrict with FK joins

Technique	Description
Avoiding SELECT *	Only required columns fetched
Filtering Early	Push filters like status='CANCELLED' before joins
Time Filtering	created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY) for weekly analysis

---

🧾 Transactional Operations & DML Examples

- UPDATE: Changed last name of user with highest cancellation to "Reddington"
- DELETE: Removed all canceled tickets of user "Reddington"
- DELETE: Wiped all canceled tickets system-wide
- UPDATE: Reduced ticket prices of "Mahan Air" from yesterday by 10%
- SELECT: Found the most reported ticket subject and count

🛠 Future Enhancements

- Implement **Materialized Views** for heavy aggregated queries
- Introduce **partitioning** on ticket table (by date/month)
- Add **caching layer** for stored procedures with frequent access
- Use **triggers** to automatically update loyalty points post-purchase
- Add a **stored function** to estimate passenger "score" (based on loyalty, spend, cities visited, etc.)

✅ Phase Deliverables Summary

Item	Description	Status
Database Schema	Full relational schema in 3NF	✅
Index Strategy	Indexed key columns and used covering indexes	✅

Item	Description	Status
22+ Analytical Queries	Business insight queries fully implemented	✅
Stored Procedures	8+ reusable and optimized procedures	✅
Optimization Techniques	Indexing, ENUMs, join strategies, date filtering	✅
Transaction Samples	Update/Delete on conditions	✅
Full Technical Documentation	Project report with insights and logic	✅

🏁 Conclusion

This phase not only meets academic and system requirements but introduces **enterprise-level best practices** such as **stored procedure encapsulation**, **efficient query execution**, and **structured data modeling**. These techniques ensure that even large-scale datasets (e.g., 10K+ users, 100K+ tickets) remain performant under analytical workloads.